RegUQ Library

Regression Uncertainty Quantification Toolkit

Comprehensive Technical Manual & API Reference

Author: Danesh Selwal
Co-Author: Prakriti Bisht
Advisor: Dr. Mahesh Pal
Version: 0.2.0
Date: July 2026

1. Executive Summary & Design Philosophy

In traditional machine learning workflows, regression models produce single-point predictions. While point estimates are useful for locating the central tendency of a target variable, they fail to communicate the confidence of the model under variable conditions. In high-stakes domains—such as hydrology, environmental monitoring, financial forecasting, and medical diagnostics—understanding the uncertainty of a prediction is just as critical as the prediction itself.

RegUQ is an industry-grade, domain-agnostic Python library designed to democratize advanced Uncertainty Quantification (UQ) techniques. By wrapping state-of-the-art estimators and UQ frameworks in a unified, pipeline-driven interface, RegUQ simplifies the transition from point prediction to rigorous interval and distributional estimation.

Core Objectives:

- **Mathematical Rigor:** Incorporates distribution-free conformal prediction algorithms that provide formal coverage guarantees even under distribution shift.
- **Unified Interface:** Provides a simple, consistent API across disparate frameworks (such as scikit-learn, MAPIE, PUNCC, LightGBM, and PyTorch).
- **Automated Optimization:** Integrates Optuna-based Bayesian hyperparameter tuning to ensure that uncertainty bounds are not artificially inflated by poorly-fitted models.
- **Colab-First Design:** Implements automatic environment bootstrapping to seamlessly resolve package installation and dependency conflicts in cloud-based notebooks.

2. Library Architecture & Key Concepts

RegUQ is structured into independent, modular workflow phases. This section outlines the architectural layers and methods supporting the library.

2.1 Base Estimators & Model Registry

RegUQ supports nine base regression models, managed via a central model registry. This registry ensures type safety, parameter validation, and consistent instantiation. The supported models are LightGBM, XGBoost, CatBoost, NGBoost, PGBM, Random Forest, Gradient Boosting, GPBoost, and TabNet.

2.2 Quantile Regression

Quantile regression estimates the conditional quantiles of a target variable rather than the conditional mean. RegUQ fits separate models for lower and upper quantiles (commonly the 5th and 95th percentiles) using the Pinball Loss function. This produces adaptive prediction intervals where the width of the interval scales dynamically with the local noise or density of the features.

2.3 Probabilistic Forecasting

Unlike quantile regression, which predicts discrete boundaries, probabilistic forecasting models the entire conditional probability distribution. RegUQ uses NGBoost and PGBM to output parameter estimates (e.g., mean and standard deviation for a Normal distribution). This enables users to calculate arbitrary quantiles, compute cumulative probabilities, and evaluate probabilistic metrics like Negative Log-Likelihood (NLL) and Continuous Ranked Probability Score (CRPS).

2.4 Conformal Prediction (Standard & Advanced)

Conformal prediction is a framework that wraps any heuristic regression model to generate prediction intervals with distribution-free, finite-sample coverage guarantees. For a specified significance level alpha, the generated interval guarantees that the true target falls inside the interval with probability at least 1 - alpha. Standard conformal prediction (via MAPIE and PUNCC) assumes exchangeable data. Advanced conformal methods are designed for non-exchangeable, temporal, or non-stationary data (e.g., NexCP, FACI, Online Conformal, and MFCS).

2.5 Hyperparameter Optimization

To prevent poor hyperparameter selection from skewing uncertainty estimates, RegUQ features an integrated tuning phase using Optuna. The library automates search space construction and executes Bayesian optimization (TPE) with pruning to find parameters that minimize validation metrics.

2.6 Model Interpretability & Explainability

Uncertainty estimates are only trustworthy if the underlying model decision process is transparent. RegUQ integrates three major explainability libraries: SHAP (SHapley Additive exPlanations) for game-theoretic feature attributions, LIME (Local Interpretable Model-agnostic Explanations) for local surrogate models, and InterpretML (Explainable Boosting Machine) for inherently interpretable Generalized Additive Models.

2.7 Google Colab Environment Bootstrapping

Installing modern machine learning and UQ libraries in Google Colab often leads to dependency conflicts. RegUQ features a bootstrapping module that detects the Colab environment, installs the exact package versions required, and programmatically restarts the active notebook runtime to apply changes immediately.

3. Public API Reference

The primary entry points of the *reguq* library are documented below. All functions accept configuration mappings or custom dataclasses (*OutputConfig* and *SplitConfig*).

3.1 run_tuning

Performs Bayesian hyperparameter optimization for a list of target models.

```
def run_tuning(
    data: Any,
    target_col: str,
    models: list[str] | tuple[str, ...] | None = None,
    tuning_config: Mapping[str, Any] | None = None,
    output_config: OutputConfig | Mapping[str, Any] | None = None,
    split_config: SplitConfig | Mapping[str, Any] | None = None,
) -> TuningResult
```

Parameter	Type	Default	Description
data	Any	Required	Dataframe, CSV path, or dictionary with train/test paths or splits.
target_col	str	Required	Name of the target variable column.
models	list[str] | None	None	Models to tune. Defaults to all compatible models.
tuning_config	dict | None	None	Mapping of parameters for Optuna (e.g., n_trials, cv, scoring).
output_config	OutputConfig | dict	None	Output configuration for saving parameters and logging.
split_config	SplitConfig | dict	None	Splitting ratios and random state for train/test evaluation.

3.2 run_quantile

Generates conditional quantile predictions to construct prediction intervals.

```
def run_quantile(
    data: Any,
    target_col: str,
    models: list[str] | tuple[str, ...] | None = None,
    params_source: Mapping[str, Any] | None = None,
    output_config: OutputConfig | Mapping[str, Any] | None = None,
    split_config: SplitConfig | Mapping[str, Any] | None = None,
    quantiles: tuple[float, float] = (0.05, 0.95),
) -> PhaseResult
```

Parameter	Type	Default	Description
data	Any	Required	Dataframe, CSV path, or dictionary with train/test paths or splits.
target_col	str	Required	Name of the target variable column.
models	list[str] | None	None	Model identifiers to evaluate.

params_source	dict | None	None	Configuration specifying parameter loading source (e.g., 'load_or_tune').
output_config	OutputConfig | dict	None	Configuration for Excel export, plot saves, and reporting.
split_config	SplitConfig | dict	None	Train/test split rules.
quantiles	tuple[float, float]	(0.05, 0.95)	The lower and upper quantile limits. Must satisfy 0 < q_low < q_high < 1.

3.3 run_probabilistic

Executes probabilistic regression using native distributional models (e.g., NGBoost, PGBM).

```
def run_probabilistic(
    data: Any,
    target_col: str,
    models: list[str] | tuple[str, ...] | None = None,
    params_source: Mapping[str, Any] | None = None,
    output_config: OutputConfig | Mapping[str, Any] | None = None,
    split_config: SplitConfig | Mapping[str, Any] | None = None,
    alpha: float = 0.1,
) -> PhaseResult
```

Parameter	Type	Default	Description
data	Any	Required	Dataframe, CSV path, or dictionary with train/test paths or splits.
target_col	str	Required	Name of the target variable column.
models	list[str] | None	None	Probabilistic models to evaluate (e.g., ['ngboost', 'pgbm']).
params_source	dict | None	None	Source mode for model parameters.
output_config	OutputConfig | dict	None	Output export configurations.
split_config	SplitConfig | dict	None	Train/test split configuration.
alpha	float	0.1	Significance level (1 - coverage). Must satisfy 0 < alpha < 1.

3.4 run_conformal_standard

Implements standard split and cross-conformal prediction via MAPIE and PUNCC.

```
def run_conformal_standard(
    data: Any,
    target_col: str,
    models: list[str] | tuple[str, ...] | None = None,
    params_source: Mapping[str, Any] | None = None,
    conformal_config: Mapping[str, Any] | None = None,
    output_config: OutputConfig | Mapping[str, Any] | None = None,
    split_config: SplitConfig | Mapping[str, Any] | None = None,
) -> ConformalResult
```

Parameter	Type	Default	Description
data	Any	Required	Dataframe, CSV path, or dictionary with train/test paths or splits.
target_col	str	Required	Name of the target variable column.
models	list[str] | None	None	Base regressors to conformalize.
params_source	dict | None	None	Parameter resolution strategy.
conformal_config	dict | None	None	Conformal configuration dict containing alpha, methods, mapie_method, calibration_size.

output_config	OutputConfig | dict	None	Output export rules.
split_config	SplitConfig | dict	None	Splitting configuration.

3.5 run_conformal_advanced

Executes advanced conformal prediction algorithms for non-exchangeable or temporal data.

```
def run_conformal_advanced(
    data: Any,
    target_col: str,
    models: list[str] | tuple[str, ...] | None = None,
    params_source: Mapping[str, Any] | None = None,
    conformal_config: Mapping[str, Any] | None = None,
    output_config: OutputConfig | Mapping[str, Any] | None = None,
    split_config: SplitConfig | Mapping[str, Any] | None = None,
) -> ConformalResult
```

Parameter	Type	Default	Description
data	Any	Required	Input dataset structure.
target_col	str	Required	Name of the target column.
models	list[str] | None	None	Base estimators to evaluate.
params_source	dict | None	None	Model hyperparameter options.
conformal_config	dict | None	None	Settings dict: alpha, methods (e.g. cvplus, cqr, online_split), decay, n_folds, n_bootstrap.
output_config	OutputConfig | dict	None	Reporting configurations.
split_config	SplitConfig | dict	None	Train/test split parameters.

3.6 run_probabilistic_advanced

Leverages advanced deep learning and instance-based uncertainty methods (CARD, IBUG, HCM, Treeffuser).

```
def run_probabilistic_advanced(
    data: Any,
    target_col: str,
    models: list[str] | tuple[str, ...] | None = None,
    params_source: Mapping[str, Any] | None = None,
    output_config: OutputConfig | Mapping[str, Any] | None = None,
    split_config: SplitConfig | Mapping[str, Any] | None = None,
    alpha: float = 0.1,
    methods: list[str] | None = None,
    card_config: Mapping[str, Any] | None = None,
    ibug_config: Mapping[str, Any] | None = None,
    hcm_config: Mapping[str, Any] | None = None,
) -> PhaseResult
```

Parameter	Type	Default	Description
data	Any	Required	Dataset mapping or dataframe.
target_col	str	Required	Name of the target column.
models	list[str] | None	None	Base regressors (e.g., ['randomforest']).
params_source	dict | None	None	Model parameters dictionary.
output_config	OutputConfig | dict	None	Output configuration settings.

split_config	SplitConfig | dict	None	Data splits settings.
alpha	float	0.1	Miscoverage probability limit. 0 < alpha < 1.
methods	list[str] | None	None	Selected advanced methods: ['card', 'ibug', 'treeffuser', 'hcm'].
card_config	dict | None	None	Deep learning settings for CARD (epochs, T, hidden_dim, n_samples).
ibug_config	dict | None	None	Neighbor lookup configuration for IBUG (n_neighbors).
hcm_config	dict | None	None	Training settings for Hyperspherical Confidence Mapping.

3.7 run_explainability

Generates global and local explanations for the models.

```
def run_explainability(
    data: Any,
    target_col: str,
    models: list[str] | tuple[str, ...] | None = None,
    params_source: Mapping[str, Any] | None = None,
    output_config: OutputConfig | Mapping[str, Any] | None = None,
    split_config: SplitConfig | Mapping[str, Any] | None = None,
    methods: list[str] | None = None,
    shap_config: Mapping[str, Any] | None = None,
    lime_config: Mapping[str, Any] | None = None,
) -> PhaseResult
```

Parameter	Type	Default	Description
data	Any	Required	Dataset structure.
target_col	str	Required	Name of the target column.
models	list[str] | None	None	Models to explain.
params_source	dict | None	None	Base model hyperparameters.
output_config	OutputConfig | dict	None	Export configuration.
split_config	SplitConfig | dict	None	Train/test split rules.
methods	list[str] | None	None	Selected methods. Default: ['shap']. Options: ['shap', 'lime', 'interpret'].
shap_config	dict | None	None	Parameters for SHAP (e.g., max_samples).
lime_config	dict | None	None	Parameters for LIME (e.g., num_features, num_samples).

3.8 run_from_config

Orchestrates a multi-phase pipeline execution from a YAML file or configuration dictionary.

```
def run_from_config(config_or_path: Mapping[str, Any] | str | Path) -> PipelineRunResult
```

Parameter	Type	Default	Description
config_or_path	str | Path | dict	Required	Path to the YAML configuration file, or dictionary structure with configuration.

4. Code Examples & Walkthroughs

This section provides complete, self-contained Python examples demonstrating each major workflow path.

4.1 Basic Quantile Regression Pipeline

Fit standard tree-based models and estimate conditional 5th and 95th percentiles.

```
import pandas as pd
from reguq import run_quantile

# 1. Prepare synthetic dataset
train_df = pd.DataFrame({
    "feature1": [1.0, 2.0, 3.0, 4.0, 5.0] * 20,
    "feature2": [-1.0, 0.5, 1.2, -0.4, 0.9] * 20,
    "target": [2.5, 4.1, 5.8, 7.2, 9.4] * 20
})
test_df = pd.DataFrame({
    "feature1": [1.5, 3.5, 4.5],
    "feature2": [0.0, 1.0, -0.5],
    "target": [3.3, 6.5, 8.0]
})

# 2. Run quantile regression
result = run_quantile(
    data={"train": train_df, "test": test_df},
    target_col="target",
    models=["lightgbm", "xgboost"],
    params_source={"mode": "defaults"},
    quantiles=(0.05, 0.95),
    output_config={
        "output_dir": "./outputs/quantile_regression",
        "export_excel": True,
        "export_plots": True
    }
)

print("Quantile Regression Complete.")
print("Metrics Dataframe:")
print(result.metrics)
```

4.2 Comprehensive Tuning & Conformal Verification

Optimize model hyperparameters via Optuna and verify predictions with standard conformal methods.

```
from reguq import run_tuning, run_conformal_standard

data_source = {
    "train_path": "Data_folder/Data/train.csv",
    "test_path": "Data_folder/Data/test.csv"
}

# 1. Optimize hyperparameters
tuning_res = run_tuning(
    data=data_source,
    target_col="target",
    models=["lightgbm"],
    tuning_config={
```

```
        "n_trials": 20,
        "cv": 3,
        "scoring": "neg_root_mean_squared_error"
    },
    output_config={"output_dir": "./outputs/tuning"}
)

# 2. Run Conformal Prediction using tuned parameters
conformal_res = run_conformal_standard(
    data=data_source,
    target_col="target",
    models=["lightgbm"],
    params_source={
        "mode": "load_only",
        "params_dir": "./outputs/tuning"
    },
    conformal_config={
        "alpha": 0.1,
        "methods": ["mapie", "puncc"]
    },
    output_config={
        "output_dir": "./outputs/conformal",
        "export_excel": True
    }
)

print("Tuning and Conformal execution finished.")
```

4.3 Advanced Probabilistic Inference (CARD/IBUG/HCM)

Run state-of-the-art deep regression diffusion models (CARD) and instance-based uncertainty estimators.

```
from reguq import run_probabilistic_advanced

# Advanced probabilistic estimation requires specific neural/boosting configurations
result = run_probabilistic_advanced(
    data="Data_folder/Data/train.csv",
    target_col="target",
    models=["randomforest"],
    methods=["card", "ibug", "hcm"],
    alpha=0.1,
    card_config={
        "epochs": 10,
        "hidden_dim": 64,
        "T": 20
    },
    ibug_config={
        "n_neighbors": 30
    },
    output_config={
        "output_dir": "./outputs/advanced_probabilistic",
        "export_excel": True,
        "save_json": True
    }
)

print("Advanced Probabilistic Run Complete.")
```

4.4 Config-Driven (YAML) Batch Execution

Configure and execute the entire workflow through a single file representation.

```
import yaml
from reguq import run_from_config

# 1. Define configuration dict (could be loaded from config.yaml)
config_data = {
    "data": {
        "train_path": "Data_folder/Data/train.csv",
        "test_path": "Data_folder/Data/test.csv",
        "target_col": "target"
    },
    "models": ["lightgbm", "xgboost"],
    "phases": ["tuning", "quantile", "conformal_standard"],
    "tuning": {
        "n_trials": 15,
        "cv": 3
    },
    "output": {
        "output_dir": "./outputs/batch_run",
        "export_excel": True,
        "export_plots": True,
        "embed_excel_charts": True
    }
}

# Save configuration to file
with open("pipeline_config.yaml", "w") as f:
```

```
    yaml.dump(config_data, f)

# 2. Run the pipeline
pipeline_result = run_from_config("pipeline_config.yaml")
print(f"Batch Run Completed. Run ID: {pipeline_result.run_id}")
```

4.5 Explainability Generation (SHAP & LIME)

Evaluate feature contribution values globally and inspect local prediction layouts.

```
from reguq import run_explainability

result = run_explainability(
    data="Data_folder/Data/train.csv",
    target_col="target",
    models=["lightgbm"],
    methods=["shap", "lime"],
    shap_config={"max_samples": 50},
    lime_config={"num_features": 5, "num_samples": 100},
    output_config={
        "output_dir": "./outputs/explainability",
        "export_plots": True
    }
)

print("Explainability metrics generated successfully.")
```

4.6 Google Colab Environment Setup

Prepare a clean environment on a fresh Google Colab notebook instance.

```
# Insert this cell block at the very top of your Colab Notebook
try:
    import reguq
except ImportError:
    # 1. Trigger Colab environment setup
    # This will clone the repository, install dependencies, and restart runtime.
    import os
    print("Installing RegUQ dependencies on Colab...")

    # 2. Bootstrap utility from reguq
    # If the package is not installed, install it from GitHub directly first
    os.system('pip install "git+https://github.com/DaneshSelwal/reguq.git"')

    from reguq.colab import bootstrap_colab_environment
    bootstrap_colab_environment(
        repo_url="https://github.com/DaneshSelwal/reguq.git"
    )
```